Linux File System Explorer Using System Calls

A Menu-Driven Linux Application in C

Project Report submitted for the partial fulfilment of grade for the course

Operating System and System Programming (25CS2104E)

in

A.Y. 2026-2027

Submitted by
Sindhuja (2520080045)
Jhanasri (2520090121)
Alekhya (2520090126)

Team Number: 08

Department: CS&IT, AI&DS

Under the guidance of

Raghupathi M

CSE Department



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY / ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Project Report entitled "**Linux File System Explorer Using System Calls**" is a record of Bonafide work of **Sindhuja – 2520080045, Jhanasri – 2520090121, Alekhya – 2520090126** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology / Artificial Intelligence & Data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Sindhuja – 2520080045

Jhanasri – 2520090121

Alekhya – 2520090126

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY / ARTIFICIAL INTELLIGENCE & DATA SCIENCE



CERTIFICATE

This is to certify that the project based report entitled "**Linux File System Explorer Using System Calls**" is a bonafide work done and submitted by **Sindhuja – 2520080045, Jhanasri – 2520090121, Alekhya – 2520090126** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology / Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our guide, **Raghupathi M**, CSE Department, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to **Dr. P. Venkateshwar Rao (CSE), Dr. Khaja Shareef (CSIT) and Dr. Sreenath Itikela (AI&DS)**, Heads of the Departments of Computer Science and Engineering, Computer Science and Information Technology, and Artificial Intelligence and Data Science respectively, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	17
8.	Conclusion & Future Scope	19
9.	References	20

1.ABSTRACT

The Linux File System Explorer is a menu-driven, command-line application written in C that lets users manage files and directories in a Linux environment through a simple interactive menu. Users can create, write, read and delete files, create and remove directories, change and display the working directory, view file information and reposition file offsets using lseek(). Every operation is carried out through Linux system calls and POSIX interfaces, which shows how a user-space program requests services from the Linux kernel rather than touching the hardware directly.

Beyond file and directory management (menu options 1–11), the application contains dedicated modules for process management (option 12), inter-process communication using anonymous pipes, named pipes (FIFOs) and signals (options 13–15), memory management (option 16) and thread synchronization (option 17). The project is built with GCC and GNU Make, and together these modules give practical exposure to all six course outcomes of Operating System and System Programming (25CS2104E). The primary focus of the project is CO-5: file systems, file abstractions and file I/O in Linux.

Team Name: Team 08 – Linux File System Explorer

Team Members:

1. Sindhuja, 2520080045
2. Jhanasri, 2520090121
3. Alekhya, 2520090126

2. INTRODUCTION

An operating system hides the complexity of hardware behind a small set of abstractions, and the file is one of the most important of them. A file system allows the operating system to store, name, organise and retrieve data in files and directories. Students usually meet these ideas as theory—kernel mode, system calls, file descriptors—and find it difficult to connect them with real programs. This project bridges that gap with a working Linux application whose every menu option is a visible request to the operating system.

2.1 Background

- **Linux file systems** organise files and folders for storage and retrieval; directories hold names that refer to files.
- **User-space applications** such as our explorer run with limited privilege and cannot access hardware directly.
- **The Linux kernel** performs privileged file operations on behalf of programs.
- **System calls** are the interface through which applications request kernel services, for example open(), read() and write().
- **File descriptors** are small integer handles that identify the files a process has open.
- **File handling and directories** cover creating, reading, writing and deleting files and managing directories.

2.2 Motivation

A simple, menu-driven tool makes it easy to try each operation, watch its effect on the file system, and relate it to the system call behind it. The additional modules extend the same idea to processes, IPC, memory and threads, so that the whole syllabus is covered by one coherent program written in C with standard Linux tools.

2.3 Organisation of the Report

Section 3 surveys related work. Section 4 states the problem and objectives. Section 5 lists the hardware and software requirements. Section 6 describes the architecture, modules, system calls and course-outcome mapping. Section 7 presents testing and results, and Section 8 gives the conclusion and future scope. References are listed in Section 9.

3. LITERATURE SURVEY

This section reviews existing tools and teaching systems that deal with the same area—file and directory management on Unix-like systems and the teaching of operating-system concepts through system calls.

1. GNU Core Utilities

The standard collection of basic Unix tools such as ls, cp, mv, rm, mkdir and cat. Each utility is a small C program that performs one file or directory task through system calls. They are the everyday reference for how file operations behave, but they present no menu or learning view of the calls involved.

URL: https://www.gnu.org/software/coreutils/

2. GNU Midnight Commander

A full-screen, text-mode file manager for Unix-like systems that offers a two-panel view for browsing, copying, moving and deleting files. It is feature-rich and aimed at productive daily use, so its internals are far more complex than a student can study end to end.

URL: https://midnight-commander.org/

3. ranger

A console file manager with Vi-style key bindings and file previews, written in Python. It shows how a terminal interface can make directory navigation efficient, but being written in a high-level language it hides the underlying system calls.

URL: https://github.com/ranger/ranger

4. nnn

A small and fast terminal file manager written in C that focuses on a minimal footprint and quick navigation. It demonstrates C-based directory handling in a real tool, yet it is designed as a utility rather than a teaching aid.

URL: https://github.com/jarun/nnn

5. BusyBox

Combines compact versions of many common Unix utilities into a single executable, widely used in embedded Linux. It shows that file-management tools can be built from a small set of system interfaces, but it does not explain them.

URL: https://busybox.net/

6. xv6 (MIT)

A simple teaching operating system that re-implements Unix Version 6 and is used to explain processes, pipes, the file system and system calls from the inside. It is excellent for kernel-level learning, whereas our project works from the user-space side on a real Linux system.

URL: https://github.com/mit-pdos/xv6-public

4. Problem Statement and Objectives

Problem Statement

Learners find it hard to connect operating-system theory with real Linux programs that perform file and directory operations through system calls.

Problem Description

File and directory handling in Linux requires an understanding of system calls, file descriptors and the kernel. A simple menu-driven Linux application is needed to perform these operations and show how user programs interact with operating-system services.

Objectives

The project was initiated to give students a practical, hands-on demonstration of operating-system services. Its objectives are:

1. Understand the role of the operating system in providing services to applications.
2. Learn how user-space programs interact with the Linux kernel.
3. Perform file and directory operations using Linux interfaces.
4. Understand file metadata and file positioning.
5. Explore process management and process control.
6. Demonstrate inter-process communication using pipes, FIFOs and signals.
7. Explore memory-management concepts.
8. Understand thread synchronization and concurrency.
9. Apply C programming and Linux systems programming concepts in a practical project.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or above
RAM	4 GB or above
Storage	10 GB free space

Table 5.1: Hardware requirements

Software Requirements

Software	Requirement
Operating System	Linux / Ubuntu
Programming Language	C Programming
Compiler	GCC Compiler
APIs	Linux / POSIX System Calls
Development Tool	VS Code / Linux Terminal
Build Tool	GNU Make
Version Control	Git and GitHub

Table 5.2: Software requirements

Ubuntu on Windows Subsystem for Linux (WSL) can also be used as a Linux development environment. The installed tools can be checked with the commands gcc --version and make --version.

6. IMPLEMENTATION

6.1 System Architecture

The explorer runs in user space. A user chooses a menu option from the command-line interface, the application performs the request by calling Linux system calls or POSIX APIs, and the Linux kernel carries out the privileged work on the file system and storage before returning the result to be shown to the user.

User space	**USER**
	COMMAND LINE INTERFACE
	FILE SYSTEM EXPLORER APPLICATION
	FILE AND DIRECTORY OPERATIONS
System-call interface	**LINUX SYSTEM CALLS / POSIX APIs**
Kernel space	**LINUX KERNEL**
	FILE SYSTEM / STORAGE
User space	**OUTPUT TO USER**

Figure 6.1: Layered architecture of the Linux File System Explorer

Typical system calls behind each group of menu operations are shown below (see src/filesystem/).

Group	System calls	Used by
File I/O	open(), read(), write(), close(), lseek()	Create, Write, Read File; File Position
Directory operations	mkdir(), rmdir(), opendir(), readdir()	Create / Remove Directory; List Directory
Metadata and deletion	stat(), unlink()	View File Information; Delete File

Table 6.1: System-call groups

6.2 Project Structure

```
KLH_CSIT-AIDS_2026-27_Team08_LinuxFileSystemExplorer/
├── PROJECT_LinuxFileExplorer/   README.md
├── data/  docs/  reports/  results/
└── src/
    ├── Makefile,  main.c,  file_explorer (executable)
    ├── filesystem/   directory_operations.c  file_info.c  file_operations.c
    ├── include/      directory_operations.h  fifo_operations.h  file_info.h
    │                 file_operations.h  memory_operations.h  pipe_operations.h
    │                 process_operations.h  signal_operations.h  thread_operations.h
    ├── ipc/          fifo_operations.c  fifo_receiver.c  fifo_sender.c
    │                 pipe_operations.c  signal_operations.c
    └── memory/       memory_operations.c
```

Figure 6.2: Repository layout (main files recorded during development)

6.3 Modules and Features

Module	Menu	Description	Source
File and Directory Management	1 – 11	List, create, write, read and delete files; create, remove and change directories; file information; file positioning with lseek().	src/filesystem/
Process Management	12	Dedicated module demonstrating process-related operating-system concepts.	process_operations.h
IPC – Pipe	13	Anonymous pipe: unidirectional channel between related processes.	src/ipc/pipe_operations.c
IPC – FIFO	14	Named pipe: a channel in the file system usable by unrelated processes (sender and receiver).	fifo_operations.c, fifo_sender.c, fifo_receiver.c
IPC – Signal	15	Asynchronous notifications delivered to a process.	src/ipc/signal_operations.c
Memory Management	16	Dedicated module for exploring memory-related concepts.	src/memory/memory_operations.c
Thread Synchronization	17	Dedicated module for thread-related functionality and concurrency.	thread_operations.h

Table 6.2: Project modules

File and directory operations (menu 1 – 11)

Menu	Feature	Purpose
1	List Directory	Display directory entries
2	Create File	Create a new file
3	Write File	Write data to a file
4	Read File	Read file contents
5	Create Directory	Make a new directory
6	Remove Directory	Remove an empty directory
7	View File Information	Show file metadata
8	Delete File	Remove a file
9	Show Current Directory	Display the working directory
10	Change Directory	Navigate to another directory
11	File Position (lseek)	Move the file offset

Table 6.3: File and directory management features

Application menu. On start-up the program displays the following menu; option 18 exits the application.

```
========================================
       LINUX FILE SYSTEM EXPLORER
========================================
1.  List Directory              10. Change Directory
2.  Create File                 11. File Position using lseek()
3.  Write File                  12. Process Management
4.  Read File                   13. IPC - Pipe Communication
5.  Create Directory           14. IPC - FIFO Communication
6.  Remove Directory           15. IPC - Signal Communication
7.  View File Information        16. Memory Management
8.  Delete File                 17. Thread Synchronization
9.  Show Current Directory     18. Exit
========================================
```

Figure 6.3: Application main menu

6.4 Linux System Calls Used

System call	Purpose	Menu operation
open()	Open or create a file	Create / Write / Read File
read()	Read data from a file descriptor	Read File
write()	Write data to a file descriptor	Write File
close()	Release a file descriptor	File operations
lseek()	Reposition the file offset	File Position
mkdir()	Create a directory	Create Directory
rmdir()	Remove an empty directory	Remove Directory
opendir()	Open a directory stream	List Directory
readdir()	Read directory entries	List Directory
stat()	Retrieve file metadata	View File Information
unlink()	Remove a file name	Delete File

Table 6.4: System calls mapped to menu operations

The mapping above follows the menu operations; the exact calls used by each function are those in src/filesystem/. Process-related calls (fork, exec, wait) and specific synchronization primitives (mutexes, condition variables, semaphores) are claimed only where they appear in the source code.

6.5 Course Outcome Mapping

CO	Course outcome	Project module	Relevance
CO-1	The OS as a Service Layer	File and directory operations through OS services	Foundational
CO-2	Processes and Process Control	Process Management (menu 12)	Additional module
CO-3	Inter-Process Communication	Pipes, FIFOs and signals (menu 13 – 15)	Additional module
CO-4	Memory Management	Memory Management (menu 16)	Additional module

CO	Course outcome	Project module	Relevance
CO-5	File Systems, File Abstractions and File I/O in Linux	File System Explorer (menu 1 – 11): file I/O, directory operations, metadata, lseek()	Primary focus
CO-6	Concurrency and Synchronization	Thread Synchronization (menu 17)	Additional module

Table 6.5: Course outcome coverage

6.6 Build and Run

```
git clone
https://github.com/Jhanasri-thota/KLH_CSIT-AIDS_2026-27_Team08_LinuxFileSystemExplor
er.git
cd KLH_CSIT-AIDS_2026-27_Team08_LinuxFileSystemExplorer/src
make
./file_explorer
```

Figure 6.4: Commands to clone, compile and run the project

6.7 Module Code

Source code of four key modules is included below.

Module 1: src/filesystem/file_operations.c

```
/* src/filesystem/file_operations.c */
/* File operations: create, write, read, delete and lseek() */

/* >>> Paste the source code of this module here <<< */
```

Module 2: src/filesystem/directory_operations.c

```
/* src/filesystem/directory_operations.c */
/* Directory operations: list, create, remove, change directory */

/* >>> Paste the source code of this module here <<< */
```

Module 3: src/filesystem/file_info.c

```
/* src/filesystem/file_info.c */
/* File information using stat() */

/* >>> Paste the source code of this module here <<< */
```

Module 4: src/ipc/pipe_operations.c

```
/* src/ipc/pipe_operations.c */
/* Anonymous pipe communication */

/* >>> Paste the source code of this module here <<< */
```

7. RESULTS & DISCUSSION

7.1 Testing and Validation

Each implemented menu option was tested by checking its behaviour against the expected result. Tests should be run only in a designated test directory so that important files are never deleted.

Test area	Validation approach
File creation	Create a file and confirm it exists
File writing and reading	Write content, check the file, read it back
File deletion	Delete a test file and confirm removal
Directory management	Create, navigate and remove directories
File metadata	Display information of an existing file
File position	Move the offset with lseek() and read from the new position
IPC	Verify pipe, FIFO and signal behavior
Process management	Test implemented process operations
Memory management	Test implemented memory functionality
Thread synchronization	Test implemented thread functionality

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Linux File System Explorer shows how a user-space program uses Linux system calls to reach operating-system services. It gives hands-on practice in file and directory management, file metadata and file positioning, and extends to process management, IPC using pipes, FIFOs and signals, memory management and thread synchronization. In terms of the objectives, the application performs file and directory operations through Linux interfaces and demonstrates the other concepts through dedicated modules, while building practical skills in C, GCC, GNU Make and Git for systems programming.

8.2 Future Scope

The following improvements are possible; they are potential enhancements and are not part of the current implementation.

Near term

- Recursive directory exploration
- File searching and filtering
- File permission management
- Improved metadata display

Medium term

- More detailed process monitoring
- Additional IPC demonstrations
- Expanded memory-management demonstrations
- Improved thread synchronization examples

Long term

- Automated testing
- Memory debugging and performance analysis